Programmable Data Plane Based ARP Flood Detection using P4

Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

with specialization in Information Technology

Submitted by

Abhishek Kumar (IIT2023261)
Piyush Chauhan (IIT2023221)
Ayush Kumar (IIT2023257)

Under the Supervision of

Dr. _____

DEPARTMENT OF INFORMATION TECHNOLOGY
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, ALLAHABAD

Prayagraj — 211015, Uttar Pradesh, India

2026

Table of Contents

List of Figures 4

List of Tables 5

Abstract 6

1 Introduction and Problem Statement 7
1.1 Overview of ARP 7
1.2 The Problem: ARP Flood Attacks 7
1.3 Limitations of Existing Detection Approaches 7
1.4 Research Motivation and Objective 8

2 Literature Survey 9
2.1 Ahuja et al. (2022) — ML-based ARP Detection in SDN 9
2.2 Aji et al. (2025) — Lightweight Rule-Based TDCM Detection 9
2.3 Roshani & Nobakht (2022) — HybridDAD P4 + ML for DDoS 9
2.4 Pešović et al. (2024) — Broadcast Storm Detection 10
2.5 Ma et al. (2016) — Bayes-based ARP Detection in SDN 10
2.6 Kumar & Dash (2024) — ML Algorithm Comparison for ARP 10
2.7 Summary and Research Gap 11

3 Methodology 12
3.1 Network Topology Design 12
3.2 P4 Data Plane Implementation 12
3.2.1 ARP Parsing and Classification 13
3.2.2 Register-Based Counters 13
3.2.3 Sampling via I2E Cloning 13
3.2.4 CPU Header Format 13

3.3 Python Controller 14
3.3.1 Switch-to-Controller Communication 14
3.3.2 Controller-to-Switch Communication 14
3.3.3 Time-Window Aggregation 14

3.4 Feature Extraction 15

3.5 Dataset Generation 16

3.6 ML Classification 16

4 Expected Results and Outcomes 18
4.1 Functional System 18
4.2 Labeled Dataset 18
4.3 ML Classification Accuracy 18
4.4 Controller Overhead Elimination 18
4.5 Detection Latency 19
4.7 Architectural Contribution 19

References 20

List of Figures

3.1 Topology…… 12

3.2 System Architecture.....15

List of Tables

2.1 Summary of Related Work 11

3.1: P4 Registers stored at each BMv2 switch……..... 13

3.2: 18 ML Features computed per 3-second time window 18

ABSTRACT

Address Resolution Protocol (ARP) is a fundamental Layer 2 protocol used in every Ethernet network to map IP addresses to MAC addresses. However, ARP was designed without any authentication or security mechanism, making it inherently vulnerable to flooding and spoofing attacks. An ARP flood attack, wherein an attacker sends thousands of forged ARP packets per second, exhausts switch ARP tables, congests network bandwidth, and can cause complete denial of service for all connected hosts.

Existing detection methods fall short in two ways: rule-based methods rely on fixed thresholds and fail under low-volume or adaptive attacks, while ML-based SDN/OpenFlow methods route every ARP packet to a central controller, creating a bottleneck that is overwhelmed by the very attack it attempts to detect. Furthermore, traditional switches are passive forwarders with no ability to monitor or classify traffic internally.

This project addresses these limitations by embedding ARP flood monitoring directly inside the data plane of P4 programmable **BMv2** switches. A hierarchical tree topology of 16 switches and 80 hosts is constructed in **Mininet**. The P4 program tracks ARP traffic using 6 register counters and uses Ingress-to-Egress (I2E) cloning to sample every 10th ARP packet, forwarding a 41-byte cpu_header to a Python controller via virtual Ethernet pairs. The controller aggregates data across 3-second time windows and computes 18 behavioral ML features — including entropy, request-reply ratio, and IP-MAC consistency. The resulting labeled dataset is used to train a Random Forest or XGBoost classifier for binary classification of normal versus flood attack traffic.

The proposed architecture eliminates the controller bottleneck, enables distributed detection across all switches simultaneously, and generates a feature-rich dataset that captures behavioral signatures of ARP floods unavailable from per-packet inspection alone.

Keywords: ***ARP Flood Detection, P4 Programming, Programmable Data Plane, BMv2, Mininet, Machine Learning,Feature Extraction, Time-Window Aggregation***

1. Introduction and Problem Statement

1.1 Overview of ARP

The Address Resolution Protocol (ARP), defined in RFC 826 [8], is a foundational Layer 2 protocol used in every Ethernet-based local area network. Its sole purpose is to resolve an IP address into its corresponding MAC address, enabling a device to construct the Ethernet frame necessary to deliver a packet to its next-hop destination. Every device on a LAN performs ARP look ups before any communication, making ARP traffic ubiquitous in any operational network.

ARP operates through a simple broadcast-request / uni-cast reply mechanism. A host broadcasts 'Who has IP X? Tell IP Y', and the owner of X replies with its MAC address. The requesting host caches this mapping in its ARP table for future use. This design was appropriate for small, trusted networks in 1982, but it contains no authentication, no integrity check, and no verification that the responding host genuinely owns the claimed IP address.

1.2 The Problem: ARP Flood Attacks

The absence of authentication in ARP enables a class of attacks known as ARP flooding. An attacker on the same Layer 2 network segment can send thousands of forged ARP packets per second with spoofed IP and MAC fields. This causes several serious consequences:

1. Switch ARP tables are exhausted by fake IP-MAC mappings, causing valid entries to be evicted.
2. Network bandwidth is congested by broadcast ARP storms consuming link capacity.
3. Denial-of-Service (DoS) is induced for all connected hosts on the affected subnet.
4. Man-in-the-Middle (MITM) attacks are enabled by poisoning ARP caches with attacker-controlled MAC addresses.

In enterprise and campus networks with hundreds of hosts, a single compromised machine can bring down an entire subnet within seconds, making ARP flood detection a critical network security requirement.

1.3 Limitations of Existing Detection Approaches

Two broad categories of ARP flood detection have been explored in prior work, each with fundamental limitations:

Rule-Based Methods: Methods such as TDCM (Aji et al., 2025) detect floods using fixed thresholds on packet rate and IP-MAC binding violations. These fail on

low-volume attacks (9% false-negative reported at 150 pps), cannot adapt to evolving attack patterns, and operate outside the network forwarding device — they are post-hoc analysis tools, not in-network sensors.

ML-Based SDN/OpenFlow Methods: Methods such as Ahuja et al. (2022) use ML classifiers (CNN-LSTM) trained on per-packet features collected via OpenFlow PacketIn messages. Every ARP packet is forwarded to a centralized Ryu controller for feature extraction and classification. Under an ARP flood (10,000+ pps), the controller itself becomes overwhelmed — the detection system collapses under the load of the attack it is designed to catch. Additionally, per-packet features miss temporal behavioral signals such as entropy trends, request-reply ratio, and IP-MAC consistency that are diagnostic of flood conditions.

1.4 Research Motivation and Objective

The fundamental gap in existing work is that switches have always been passive forwarders. All monitoring logic sits outside the switch, requiring traffic to be copied or redirected to an external device. With the emergence of P4 programmable switches [7], it is now possible to embed classification, counting, and sampling logic directly in the data plane at line rate.

This work is motivated by the need for a detection architecture that: (1) operates entirely inside the switch data plane without depending on controller availability, (2) samples traffic efficiently without forwarding every packet externally, (3) generates rich behavioral features across time windows, and (4) scales to a distributed multi-switch topology without centralized polling.

The objective of this project is to design, implement, and evaluate a P4-based ARP flood detection system on a 16-switch, 80-host Mininet topology, generate a labeled dataset of 18 behavioral features per 3-second time window, and train an ML classifier to distinguish normal from flood attack traffic.

2. Literature Survey

This section reviews six related works in the areas of ARP attack detection, SDN-based network monitoring, P4 programmable switches, and ML-based intrusion detection. The papers are presented in decreasing order of relevance to our work.

2.1 Ahuja et al. (2022) — ML-based ARP Detection in SDN [1]

Ahuja et al. proposed a CNN-LSTM based ARP attack detection system deployed on a Mininet topology (depth 3, fanout 3, approximately 13 hosts) using the Ryu SDN controller and OpenFlow. Their system collects 17 per-packet features via OpenFlow PacketIn messages and achieves 99.73% classification accuracy. This is the most closely related work to ours in terms of problem domain.

However, their approach has critical architectural limitations: the OpenFlow PacketIn mechanism sends every ARP packet to the controller for processing. Under flood conditions (10,000+ pps), this creates a controller bottleneck — the detection pipeline is overwhelmed by the very event it is designed to detect. Their features are per-packet only (raw header fields), missing temporal behavioral signals. Their topology is small (13 hosts) and the switch remains passive.

Our work resolves all three limitations: P4 I2E cloning samples 1-in-10 ARP packets (controller load constant regardless of flood rate), time-window aggregation captures behavioral features, and monitoring logic is embedded inside the switch itself.

2.2 Aji et al. (2025) — Lightweight Rule-Based TDCM Detection [2]

Aji et al. proposed TDCM (Time-Density-based Continuous Monitoring), a lightweight rule-based method that combines packet rate thresholds with IP-MAC binding verification for ARP flood detection in office networks. Their evaluation is conducted purely in Python simulation without any real network or programmable switch.

Key limitations include: detection only effective at high flood rates (9% false-negative at 150 pps), no ML adaptability, operates outside any network device, and uses only 2 features. Our work uses real P4/BMv2 switches, 18 features, and a supervised ML classifier trained on collected data.

2.3 Roshani & Nobakht (2022) — HybridDAD P4+ML for DDoS [3]

HybridDAD is the closest architectural paper to our work. Roshani and Nobakht

proposed embedding feature extraction in P4 programmable switches with external ML classification (RF, KNN, SVM, RNN) using sliding window aggregation. They target IP-layer DDoS attacks (TCP/UDP/ICMP/DHCP floods) and achieve 99.9%+ accuracy with Random Forest at T=2 seconds.

Our work draws architectural inspiration from HybridDAD but extends it specifically to ARP floods: we add ARP-specific features (entropy, request-reply ratio, gratuitous ARP count, IP-MAC consistency) not present in their system, scale to 16 switches (vs. single switch), and use I2E cloning with cpu_header rather than digest messages.

2.4 Pešović et al. (2024) — Broadcast Storm Detection [4]

Pešović et al. used Wireshark PCAP capture and agglomerative hierarchical clustering (MATLAB) to detect broadcast storms in LANs. Their method captures traffic from an external monitoring port and applies unsupervised clustering with 3 features (MAC address, packet count, byte count). Detection is offline, requires manual interpretation of cluster outputs, and is not ARP-specific.

In contrast, our system operates in real-time inside the switch data plane, uses supervised ML with 18 features, requires no external capture hardware, and generates automated labeled classifications.

2.5 Ma et al. (2016) — Bayes-based ARP Detection in SDN [5]

Ma et al. proposed a Bayesian probability model with 4 manually defined features for ARP attack detection using the Floodlight SDN controller. Their work targets ARP poisoning and MITM attacks in cloud VM environments (1000 hosts, flat topology). It represents a historical baseline for SDN-based ARP detection from the pre-P4 era.

Limitations: Bayesian model with manual priors is not adaptive, every packet is forwarded to the controller, and the work is from 2016 before P4 programmable switches were widely available. We cite this work as historical context for the evolution from SDN/OpenFlow to P4-based monitoring.

2.6 Kumar & Dash (2024) — ML Algorithm Comparison for ARP [6]

Kumar and Dash compared five ML algorithms (Random Forest: 94%, CNN: 91%, LSTM: 90%, SVM: 88%, Isolation Forest: 85%) on an existing ARP dataset. They found Random Forest achieved the highest accuracy with the lowest false positive and false negative rates. However, the work contributes no novel network architecture, no data collection method, and provides vague feature descriptions with

no temporal aggregation.

We cite this work specifically to justify our selection of Random Forest as the primary ML classifier, supported by their comparative empirical evidence.

2.7 Summary and Research Gap

Table 2.1 summarizes all reviewed papers. The analysis reveals a consistent gap: no existing work embeds ARP-specific monitoring logic inside a P4 programmable switch data plane, computes behavioral time-window features, and scales to a distributed multi-switch topology. Our work fills this gap.

Paper	Approach	Key Limitation
Ahuja et al. [1] (2022)	SDN/Ryu + CNN-LSTM 17 per-packet features Mininet topology	PacketIn bottleneck at flood rate Per-packet only, no windowed features Small topology (13 hosts)
Aji et al. [2] (2025)	Time-density + IP-MAC binding No ML, no training data Python simulation only	Detection improves mainly during high-rate flooding. Tested only in simulation, not real networks. Not adaptable to evolving attacks
Roshani & Nobakht [3] (2022)	P4 + RF/KNN/SVM/RNN Sliding window aggregation DDoS (TCP/UDP/ICMP/DHCP)	Targets DDoS, NOT ARP specifically No ARP-specific features Single switch scenario
Pešović et al. [4] (2024)	Wireshark PCAP capture Agglomerative clustering MATLAB Offline analysis only	Offline, not real-time Unsupervised — human interprets clusters Only 3 shallow features,broadcast storms, not specific attacks like ARP flooding.
Ma et al. [5] (2016), Tsinghua S&T	Bayesian probability model SDN OpenFlow + Floodlight Manual prior probabilities	Outdated pre-P4 era (2016) Every packet to controller Targets spoofing not flooding
Kumar & Dash [6] (2024), IJIRCST	Compares 5 ML algorithms Random Forest: 94% accuracy No novel architecture	Scalability & Controller Load No temporal aggregation

Table 2.1: Summary of Related Work

Key Research Gaps Addressed by This Work:

- No existing work embeds ARP monitoring inside P4 data plane registers
- Existing ML methods create controller bottleneck under flood conditions
- No publicly available P4-based ARP flood dataset with time-window features
- No ARP-specific entropy, req-reply ratio, or IP-MAC features in P4 systems
- Existing topologies too small (single switch or 13 hosts) for distributed sensing

3. Methodology

This chapter describes the design and implementation of the proposed ARP flood detection system. The methodology is organized into five phases: network topology design, P4 data plane implementation, Python controller development, dataset generation, and ML classification.

3.1 Network Topology Design

A hierarchical tree topology is constructed using **Mininet** with **BMv2** (simple_switch) as the P4 software switch. The topology consists of 16 switches and 80 hosts organized in three layers:

- Root Layer: 1 root switch (s11) connecting all aggregation switches
- Aggregation Layer: 5 aggregation switches (s12–s16) each connecting to 2 edge switches
- Edge Layer: 10 edge switches (s1–s10) each connecting to 8 hosts
- Hosts: 80 hosts (h1–h80), 8 per edge switch, subnet 10.0.N.0/16 per switch

Each switch has a dedicated virtual Ethernet pair (sN-cpu0 and sN-cpu1) for CPU communication. The topology uses a pure tree structure with no cycles, eliminating the need for spanning tree protocol.

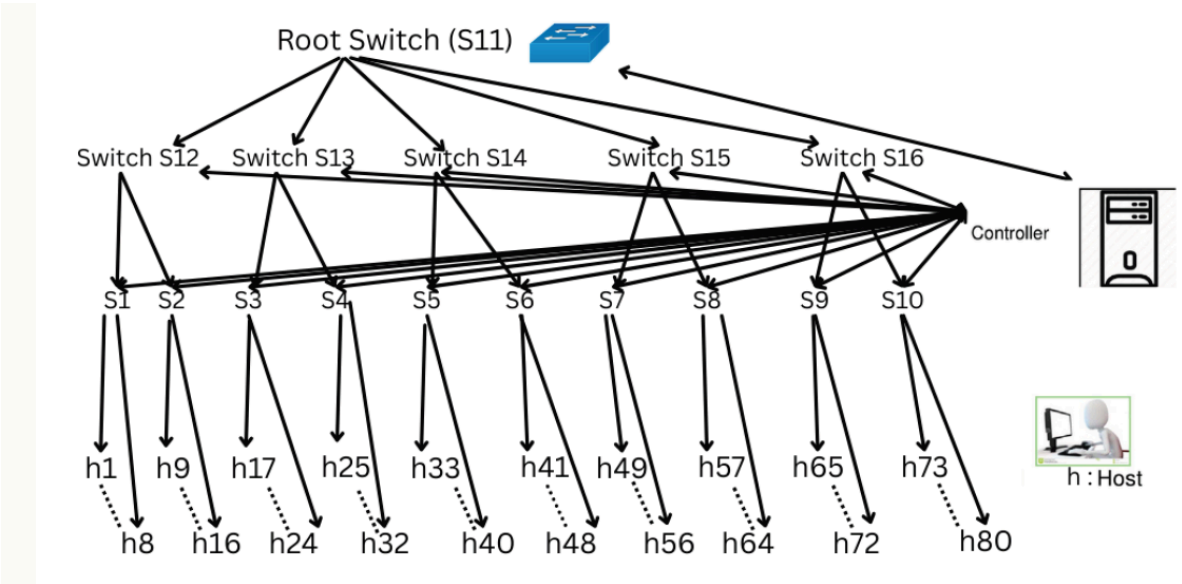

Figure 3.1: Hierarchical tree network topology used in simulation

3.2 P4 Data Plane Implementation

The P4 program arp_monitor.p4 is compiled using p4c v1.2.5.11 to produce a BMv2-compatible JSON configuration. The data plane implements the following pipeline:

3.2.1 ARP Parsing and Classification

The parser extracts Ethernet and ARP headers from incoming packets. An ARP packet is classified into one of four types based on opcode and address fields:

- Request: opcode = 1
- Reply: opcode = 2
- Gratuitous: sender IP equals target IP
- Broadcast: Ethernet destination is FF:FF:FF:FF:FF:FF

3.2.2 Register-based Counters

Six P4 registers are maintained per switch in the ingress pipeline. Each register is a single 32-bit (or 8-bit for counter) value stored at index 0:

Register	Type	Incremented When
reg_arp_total	bit<32>	Any ARP packet is received
reg_arp_request	bit<32>	ARP opcode equals 1 (Request)
reg_arp_reply	bit<32>	ARP opcode equals 2 (Reply)
reg_gratuitous	bit<32>	Sender IP address equals Target IP address
reg_broadcast	bit<32>	Ethernet destination is FF:FF:FF:FF:FF:FF
reg_sample_counter	bit<8>	Any ARP packet (cycles 0-9, resets on 10th)

Table 3.1: P4 Registers stored at each BMv2 switch

3.2.3 Sampling via I2E Cloning

On every ARP packet, reg_sample_counter is incremented. When reg_sample_counter equals 10, the ingress pipeline: (1) reads all 6 register values into packet metadata, (2) resets all registers to 0, and (3) executes clone_preserving_field_list(I2E, session=100). The original packet is forwarded normally to its destination. The clone is directed to egress where a 41-byte cpu_header is prepended, and the clone exits through port 255 into the sN-cpu0 veth interface.

3.2.4 CPU Header Format (41 bytes)

The cpu_header carries all information needed by the controller to update its aggregator state:

- reason (8 bits): clone reason identifier
- switch_id (8 bits): originating switch identifier

- ingress_port (9 bits) + pad (7 bits)
- sender_ip (32 bits), sender_mac (48 bits), target_ip (32 bits), opcode (16 bits)
- reg_arp_total, reg_arp_request, reg_arp_reply, reg_gratuitous, reg_broadcast (32 bits each)
- reg_sample_counter (8 bits)

3.3 Python Controller

The Python controller manages two communication channels with the switches and maintains a time-window aggregator for dataset generation.

3.3.1 Switch-to-Controller: I2E Clone Reception

16 Scapy sniff() threads are created at startup — one per switch CPU interface (sN-cpu1). Each thread blocks in an AF_PACKET raw socket recv() call. When a clone arrives, the OS wakes the thread, and _on_packet() is called. The 41-byte cpu_header is parsed using struct.unpack(). A threading.Lock() protects shared state.

3.3.2 Controller-to-Switch: MAC Learning

When a new sender_mac is observed, the controller installs a forwarding rule via Thrift CLI: simple_switch_CLI calls table_add tbl_l2 l2_forward <mac> => <port> over TCP port 9090+N. This is the only direction of switch-controller communication — all ARP monitoring data flows from switch to controller only.

3.3.3 Time-Window Aggregation

A background timer thread fires every 3 seconds, calling aggregator.flush() on the WindowAggregator. During the window, each ingest() call accumulates register snapshot values and digest packet fields (sender_ip, sender_mac, target_ip, opcode). On flush(), all 18 ML features are computed and written as one CSV row with a timestamp and label.

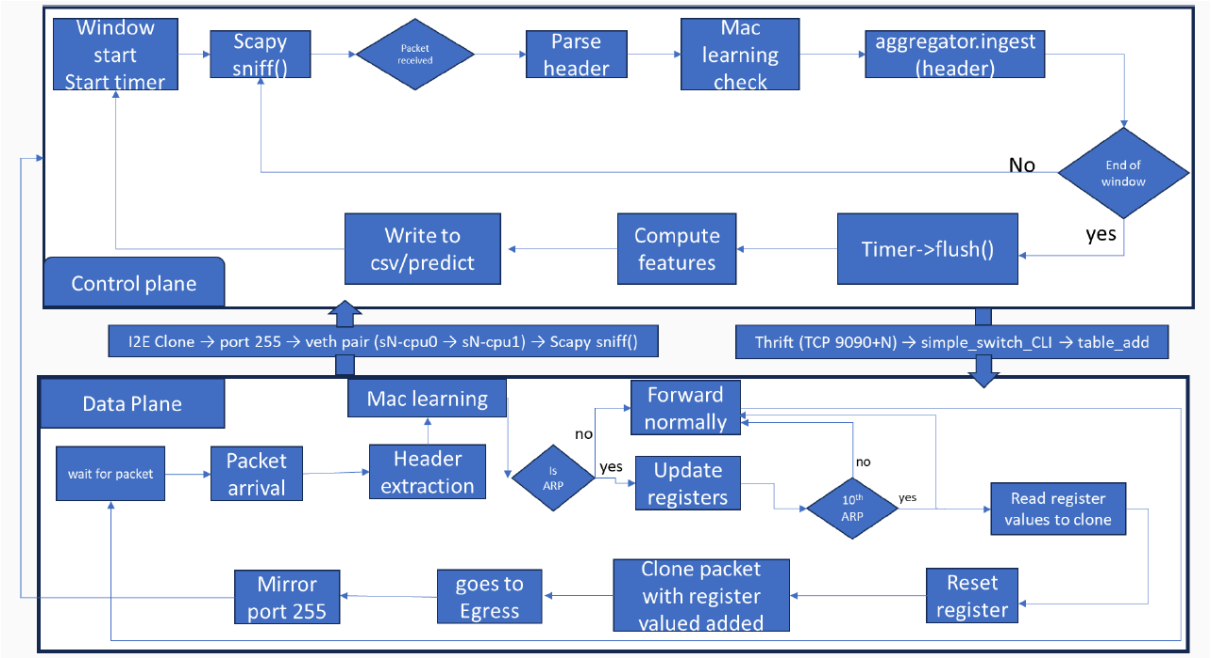

Figure 3.2: System architecture showing data plane (P4) and control plane (Python controller) interaction

3.4 Feature Extraction

On each 3-second flush(), 18 ML features are computed across three categories.

#	Feature Name	Description
F1	arp_total	Total ARP packets received in the 3-second window
F2	arp_request_count	Number of ARP Request packets (opcode=1)
F3	arp_reply_count	Number of ARP Reply packets (opcode=2)
F4	req_reply_ratio	Request count / (Reply count + epsilon)
F5	gratuitous_arp_count	Packets where sender IP equals target IP
F6	broadcast_count	Packets with Ethernet dst FF:FF:FF:FF:FF:FF
F7	packet_rate	Total ARP packets per second in window
F8	request_rate	ARP requests per second in window
F9	reply_rate	ARP replies per second in window
F10	src_ip_entropy	Shannon entropy of source IP distribution
F11	src_mac_entropy	Shannon entropy of source MAC distribution
F12	target_ip_entropy	Shannon entropy of target IP distribution
F13	unique_src_ip_count	Count of distinct source IP addresses
F14	unique_src_mac_count	Count of distinct source MAC addresses
F15	top1_ip_share	Fraction of packets from top source IP
F16	digest_sample_count	Number of sampled packets in window
F17	ip_mac_consistency_score	Fraction of IPs seen with consistent MAC address
F18	new_ip_ratio	Fraction of source IPs not seen in previous window

Table 3.2: 18 ML Features computed per 3-second time window

The features are computed as follows: Counter features (F1–F9) are derived directly from register snapshots accumulated over the window. Entropy features (F10–F12) use Shannon entropy H = -sum(p_i * log2(p_i)) over the distribution of sender IPs, sender MACs, and target IPs from digest packets. Diversity features (F13–F16) count unique addresses. Anomaly features (F17–F18) compute IP-MAC consistency by checking whether the same IP always maps to the same MAC, and new_ip_ratio by comparing current IPs against a rolling history set.

3.5 Dataset Generation

Two types of traffic are generated to create a labeled binary classification dataset:

- Normal Traffic (label=0): Slow host-to-host pings across the topology, natural ARP resolution, background traffic at 50–1000 pps per switch.
- Attack Traffic (label=1): Scapy ARP flood launched from h1: sendp(Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(op=1, pdst='10.0.0.1'), loop=1, inter=0). This generates 100,000+ ARP requests per second.

The dataset label is toggled atomically using signal.signal(SIGUSR1) without restarting the controller. The operator sends kill -USR1 <pid> to switch the running label from 0 to 1 (or back). Each 3-second window produces exactly one labeled CSV row.

3.6 ML Classification

The labeled CSV dataset is used to train and evaluate ML classifiers. The primary classifier is Random Forest, selected based on Kumar and Dash (2024) who found it achieved the highest accuracy (94%) with the lowest false positive and false negative rates among five tested algorithms. XGBoost is evaluated as a secondary classifier.

The dataset is split 80/20 train/test. Evaluation metrics include accuracy, precision, recall, F1-score, and confusion matrix. Given the strong feature separation expected between normal (low entropy, balanced rates) and attack (zero entropy, extreme request rate, zero replies) windows, accuracy exceeding 95% is anticipated.

Software Stack Summary:

- Network Simulation: Mininet, BMv2 simple_switch v1.15.0
- P4 Compilation: p4c v1.2.5.11
- Controller: Python 3, Scapy, struct, threading, csv, subprocess
- Switch Communication: Thrift TCP (9090+N), Veth pairs (sN-cpu0 ↔ sN-cpu1)
- ML: scikit-learn (Random Forest), XGBoost, pandas
- Operating System: Ubuntu 24.04 (native, no VM)

4. Expected Results and Outcomes

This chapter describes the expected outcomes of the fully realized project based on the proposed architecture, the richness of the feature set, and comparisons with existing approaches.

4.1 Functional System

The primary deliverable is a fully functional ARP flood detection system deployed on real P4/BMv2 switches inside a Mininet-emulated network. Unlike prior simulation-only work (Aji et al., 2025), all components — switches, hosts, veth pairs, controller threads, and dataset writer — operate on actual virtual networking infrastructure with real packet flows. Every switch in the 16-switch topology independently monitors its local ARP traffic at line rate without any dependency on the controller for forwarding decisions.

4.2 Labeled Dataset

The system will produce a labeled CSV dataset where each row represents a 3-second window of ARP activity across one switch. Each row contains a timestamp, 18 behavioral features, and a binary label (0 = normal, 1 = attack). The expected feature signatures are:

- Normal traffic: src_ip_entropy = 2.0–3.5 bits, req_reply_ratio ≈ 1.0, packet_rate = 50–1000 pps, ip_mac_consistency_score ≈ 1.0
- Attack traffic: src_ip_entropy ≈ 0.0 bits (single source), req_reply_ratio = infinity (zero replies), packet_rate = 50,000–100,000 pps, new_ip_ratio → 0 (same fake IPs repeated)

This clear separation in feature space is expected to yield high classification accuracy. The dataset will be the first publicly available P4-based ARP flood dataset with time-window behavioral features, filling a gap noted in the literature.

4.3 ML Classification Accuracy

Based on the richness and discriminative power of the 18 features, we expect:

- Random Forest classification accuracy > 95%
- Precision and recall both > 95% for both classes
- F1-score > 0.95 for attack class
- False positive rate < 3% (very few normal windows classified as attack)
- False negative rate < 5% (very few attack windows missed)

These expectations are grounded in the fact that entropy collapses completely to near-zero during a flood (single source), while request-reply ratio becomes infinite

(zero replies) and packet rate increases by 100x or more — three independently strong features that together create a separable classification boundary.

4.4 Controller Overhead Elimination

A key architectural result will be that controller CPU load and memory usage remain effectively constant regardless of the attack intensity. Under normal traffic (100 pps), the controller receives approximately 10 clones per second per switch. Under a flood attack (100,000 pps), the controller still receives only 10,000 clones per second per switch (1-in-10 sampling). This is a 10x reduction compared to OpenFlow PacketIn which would forward all 100,000 packets per second.

This demonstrates the key architectural advantage of P4 data plane sampling over OpenFlow-based approaches, and is the primary reason the system does not collapse under the attack conditions it is designed to detect.

4.5 Detection Latency

Attack detection latency is bounded by the window size: 3 seconds. An ARP flood attack is detectable within one 3-second window of its initiation. This is acceptable for practical deployment as ARP floods typically persist for minutes.

4.6 Architectural Contribution

The project demonstrates that P4 programmable switches can function as active security sensors rather than passive forwarders. This represents a fundamental shift in network security architecture: moving monitoring logic from external appliances and controllers into the forwarding hardware itself, at the speed of packet processing, without controller dependency for the core detection function.

References

1. Ahuja N., Singal G., Mukhopadhyay D., Nehra A. (2022). Ascertain the efficient machine learning approach to detect different ARP attacks. Computers and Electrical Engineering, Vol. 99. Elsevier. DOI: 10.1016/j.compeleceng.2022.107757
2. Aji R.F., Kurniawan H., Utami N.P. (2025). A Lightweight Rule-Based Detection Approach for ARP Flooding Malware in Office Networks. International Journal of Advanced Computer Science and Applications (IJACSA), Vol. 16, No. 12.
3. Roshani M., Nobakht M. (2022). HybridDAD: Detecting DDoS Flooding Attack using Machine Learning with Programmable Switches. Proceedings of ARES 2022, Vienna, Austria. ACM. DOI: 10.1145/3538969.3538991
4. Pešović U., Đurašević Pešović S., Savić B., Marković D. (2024). Detection of Broadcast Storms in Local Area Networks. TIE 2024, 10th International Scientific Conference on Telecommunications and Information Engineering.
5. Ma H., Ding H., Yang Y., Mi Z., Yang J.Y., Xiong Z. (2016). Bayes-Based ARP Attack Detection Algorithm for Cloud Centers. Tsinghua Science and Technology, Vol. 21, No. 1, pp. 16–27.
6. Kumar M., Dash C.S. (2024). Detecting and Preventing ARP Spoofing Attacks Using Real-Time Data Analysis and Machine Learning. International Journal of Innovative Research in Computer Science and Technology (IJIRCST), Vol. 12, Issue 5.
7. Bosshart P., Daly D., Gibb G., Izzard M., McKeown N., Rexford J., Schlesinger C., Talayco D., Vahdat A., Varghese G., Walker D. (2014). P4: Programming Protocol-Independent Packet Processors. ACM SIGCOMM Computer Communication Review, Vol. 44, No. 3, pp. 87–95.
8. Plummer D.C. (1982). RFC 826: An Ethernet Address Resolution Protocol. Internet Engineering Task Force (IETF).
9. Shin S., Yegneswaran V., Porras P., Gu G. (2013). AVANT-GUARD: Scalable and Vigilant Switch Flow Management in Software-Defined Networks. Proceedings of ACM CCS 2013.
10. P4 Language Consortium. P4 Language Specification v1.2. Available: https://p4.org/p4-spec/
11. BMv2 Behavioral Model Documentation. Available: https://github.com/p4lang/behavioral-model